

Problem Proeminența

Autor problemă: PAUL DIAC

Scurtă descriere a problemei

Se dau N altitudini ale unui teren reprezentate printr-un șir $a[0], a[1], \dots, a[N-1]$. Pentru fiecare *vârf* (o poziție i astfel încât $a[i-1] < a[i] > a[i+1]$), trebuie calculată **proeminența** sa, definită ca diferența dintre înălțimea vârfului și cea mai adâncă vale întâlnită înaintea sa. Se afișează proeminențele pentru toate vârfurile.

Observații-cheie

Observație 1. Un vârf este o poziție locală de maxim strict: $a[i-1] < a[i] > a[i+1]$.

Observație 2. Proeminența unui vârf V este $h(V) - \min\{h(\text{vale } Y) \mid Y \text{ se află în stânga lui } V\}$.

Observație 3. Când întâlnim un vârf nou V_2 mai înalt decât un vârf anterior V_1 , vârful V_1 nu mai poate fi **niciodată** cea mai înaltă stângă pentru niciun vârf din dreapta. Deci V_1 poate fi eliminat din considerație.

Observație 4. Această proprietate de eliminare sugerează utilizarea unei **stive** care menține vârfurile în ordinea întâlnirii și în ordine strict descrescătoare a înălțimilor.

Observație 5. Pentru fiecare vârf trebuie să cunoaștem cea mai adâncă vale din dreapta sa. Putem calcula aceasta în timpul parcurgerii, actualizând permanent valea minimă.

Observație 6. Dacă parcurgem șirul oglindit (de la dreapta la stânga), problema devine identică: calculăm proeminența la dreapta fiecărui vârf original. Apoi, pentru fiecare vârf, proeminența finală este $\min(\text{proeminență stânga}, \text{proeminență dreapta})$.

Soluție

Vom prezenta soluția în trei etape.

Eliminarea duplicatelor

Înainte de orice calcul, eliminăm valorile duplicate consecutive din șir:

```
for (int i = 0; i < N; i++) {
    scanf("%ld", &a[i]);
    if (i > 0 && a[i] == a[i-1]) {
        i--; N--; // eliminam duplicatele
    }
}
```

Aceasta simplifică logica, deoarece nu pot exista vârfuri sau văi între valori egale consecutive.

Cazul particular $C = 1$

Dacă $C = 1$, problema se reduce la simpla numărare a vârfurilor:

```
if (C == 1) {
    for (int i = 1; i < N-1; i++) {
        if (a[i-1] < a[i] && a[i] > a[i+1]) { s++; }
    }
    printf("%ld\n", s);
    return 0;
}
```

Cazul general: două parcurgeri

Pentru cazul general, executăm două parcurgeri cu stiva:

1. **Parcurea stânga-dreapta** ($d = 0$): calculăm proeminența la stânga pentru fiecare vârf.
2. **Oglindirea șirului și a doua parcurgere** ($d = 1$): calculăm proeminența la dreapta.

Structurile de date folosite:

- `peak[top]` — stiva de indici ai vârfurilor, menținută în ordine strict descrescătoare a înălțimilor;
- `valley[top]` — cea mai adâncă vale întâlnită în dreapta vârfului `peak[top]`;
- `prom[d][i]` — proeminența calculată în parcurgerea d pentru vârful de pe poziția i .

```
for (int d = 0; d <= 1; d++) {
    for (int i = 1; i < N-1; i++) {
        if (a[i-1] < a[i] && a[i] > a[i+1]) { // este varf
            prom[d][i] = a[i];
            while (top >= 0 && a[peak[top]] <= a[i]) {
                popTop(); // elimin varfurile mai mici din stiva
            }
            if (top >= 0 && a[peak[top]] > a[i]) {
                prom[d][i] = a[i] - valley[top];
            }
            peak[++top] = i;
            valley[top] = a[i];
        }
        if (top >= 0 && valley[top] > a[i]) {
            valley[top] = a[i]; // actualizam cea mai adanca vale
        }
    }
    // oglindim pentru a doua parcurgere
    for (int i = 0; i < N / 2; i++) {
        long aux = a[i]; a[i] = a[N-1-i]; a[N-1-i] = aux;
    }
    top = -1;
}
```

Funcția `popTop()` actualizează și cea mai adâncă vale:

```
void popTop() {
    if (top < 0) { return; }
    if (top >= 1 && valley[top-1] > valley[top]) {
        valley[top-1] = valley[top];
    }
    top--;
}
```

După cele două parcurgeri, combinăm rezultatele:

```
for (int i = 1; i < N-1; i++) if (a[i-1] < a[i] && a[i] > a[i+1]) {
    if (prom[0][i] > prom[1][N-1-i]) {
        prom[0][i] = prom[1][N-1-i];
    }
}
```

Pentru fiecare vârf, reținem **minimumul** dintre cele două proeminențe.

Complexitate

- **Timp:** $O(N)$ — fiecare element este procesat de cel mult două ori, iar fiecare vârf este adăugat și eliminat din stivă cel mult o dată.
- **Memorie:** $O(N)$ — pentru stiva de vârfuri, valea curentă și tablourile de proeminență.

Explicația corectitudinii

Lemma 1. La orice moment în parcurgere, stiva `peak[]` conține indicii vârfurilor întâlniți până acum, în ordine strict descrescătoare a înălțimilor.

Demonstrație. Când întâlnim un vârf nou de înălțime h :

- Eliminăm din stivă toate vârfurile cu înălțime $\leq h$ (prin `while`).
- Aceste vârfuri nu vor mai fi niciodată necesare, deoarece h este mai înalt și blochează vizualizarea pentru orice vârf din dreapta.

După eliminare, stiva satisface proprietatea de ordine strict descrescătoare, iar apoi adăugăm vârful nou. $\square$

Lemma 2. Pentru fiecare vârf V , `valley[idx]` asociat celui mai apropiat vârf mai înalt din stânga este cea mai adâncă vale situată între cele două vârfuri.

Demonstrație. Când adăugăm un vârf P în stivă, inițializăm `valley[top] = a[P]`, considerând propria sa poziție ca cea mai adâncă vale. La fiecare pas, actualizăm `valley[top]` cu minimumul dintre valoarea curentă și $a[i]$ dacă $a[i]$ este mai mică. Astfel, după parcurgerea tuturor elementelor dintre V și vârful mai înalt H , `valley[idx]` va fi exact minimumul altitudinilor din acel interval — cea mai adâncă vale. $\square$

Lemma 3. Proeminența calculată la stânga pentru un vârf V este corectă.

Demonstrație. Fie H primul vârf mai înalt decât V în stânga (acesta este vârful din vârful stivei după eliminări). Conform **Lemmei 2**, `valley[idx]` este cea mai adâncă vale dintre H și

V. Prin urmare, proeminența calculată $a[V] - \text{valley}[idx]$ este exact diferența dintre înălțimea vârfului și cea mai adâncă vale din stânga sa. $\square$

Teoremă. Algoritmul afișează pentru fiecare vârf proeminența corectă, definită ca $\min(\text{proeminență stânga}, \text{proeminență dreapta})$.

Demonstrație. Conform **Lemmei 3**, prima parcurgere calculează corect proeminența la stânga, iar a doua parcurgere (după oglindire) calculează corect proeminența la dreapta. Prin definiția problemei, proeminența finală este minimumul dintre acestea două, ceea ce face algoritmul corect.

$\square$